《动手学深度学习》——前言、安装、符号

内容和结构

全书大致可分为三个部分，在图1中用不同的颜色呈现：

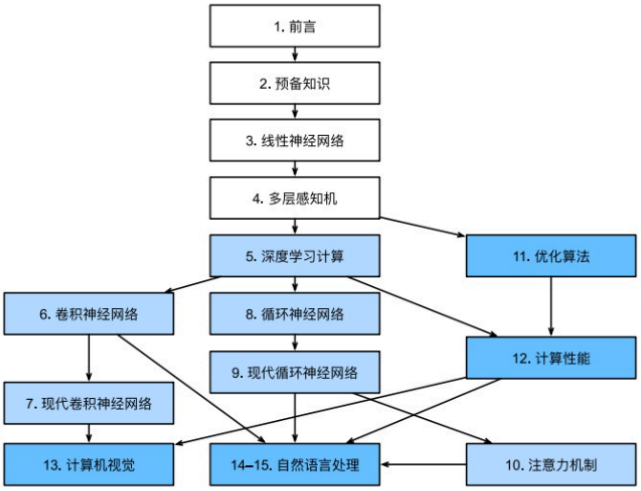

图1: 全书结构

- **第一部分包括基础知识和预备知识**。1节 提供深度学习的入门课程。然后在2节中，我们将快速介绍实践深度学习所需的前提条件，例如如何存储和处理数据，以及如何应用基于线性代数、微积分和概率基本概念的各种数值运算。3节 和4节涵盖了深度学习的最基本概念和技术，例如线性回归、多层感知机和正则化。
- **接下来的五章集中讨论现代深度学习技术**。5节 描述了深度学习计算的各种关键组件，并为我们随后实现更复杂的模型奠定了基础。接下来，在6节和7节中，我们介绍了卷积神经网络（convolutional neural network，CNN），这是构成大多数现代计算机视觉系统骨干的强大工具。随后，在8节和9节中，我们引入了循环神经网络（recurrent neural network，RNN），这是一种利用数据中的时间或序列结构的模型，通常用于自然语言处理和时间序列预测。在10节中，我们介绍了一类新的模型，它采用了一种称为注意力机制的技术，最近它们已经开始在自然语言处理中取代循环神经网络。这一部分将帮助读者快速了解大多数现代深度学习应用背后的基本工具。
- **第三部分讨论可伸缩性、效率和应用程序**。首先，在 11 节中，我们讨论了用于训练深度学习模型的几种常用优化算法。下一章 12 节将探讨影响深度学习代码计算性能的几个关键因素。在 13 节中，我们展示了深度学习在计算机视觉中的主要应

用。在 14 节和 15 节中，我们展示了如何预训练语言表示模型并将其应用于自然语言处理任务。

代码

本书中经常导入和引用的函数、类等封装在`d2l`包中。对于要保存到包中的任何代码块，比如一个函数、一个类或者多个导入，我们都会标记为`#@save`。d21软件包是轻量级的，仅需要以下软件包和模块作为依赖项：

```
#@save
import collections  # Python 内置的高级数据结构库，比普通 list/dict 更强。
import hashlib  # 哈希库，用来计算文件 / 字符串的 MD5、SHA1 等哈希值。
import math  # Python 标准数学库。
import os  # 操作系统接口库，和文件、路径、文件夹打交道。
import random  # 随机数库。
import re  # 正则表达式库，字符串匹配神器。
import shutil  # 高级文件操作库，比 os 更强。
import sys  # 系统相关库，控制 Python 解释器本身。
import tarfile  # 解压 .tar、.tar.gz 格式压缩包。
import time  # 时间库。
import zipfile  # 解压 .zip 文件。
from collections import defaultdict  # defaultdict = 自带默认值的字典。
import pandas as pd  # 最强表格数据处理库。
import requests  # HTTP 请求库，用来从网上下载东西。
from IPython import display  # Jupyter / Notebook 专用。
from matplotlib import pyplot as plt  # Python 最经典画图库。
from matplotlib_inline import backend_inline  # 让 matplotlib 图像直接嵌入 Jupyter Notebook，不弹出新窗口。

d2l = sys.modules[__name__]
```

本书中的大部分代码都是基于PyTorch的。PyTorch是一个开源的深度学习框架，在研究界非常受欢迎。本书中的所有代码都在最新版本的PyTorch下通过了测试。

下面是我们如何从PyTorch导入模块。

```
#@save
import numpy as np  # 最基础的数值计算库 → 所有机器学习的基石
import torch  # PyTorch 核心库 → 深度学习框架本体
import torchvision  # PyTorch 官方视觉工具库 → 专门处理图像任务
from PIL import Image  # Python 最经典的图像处理库
from torch import nn  # PyTorch 的神经网络模块 → 造层神器
from torch.nn import functional as F  # nn 的函数式 API → 灵活调用层与激活函数
from torch.utils import data  # PyTorch 数据加载模块 → 批量喂数据给模型
from torchvision import transforms  # 图像预处理 / 增强工具 → 图像 “整容院”
```

安装

我们需要配置一个环境来运行 Python、Jupyter Notebook、相关库以及运行本书所需的代码，以快速入门并获得动手学习经验。

安装 Miniconda

最简单的方法就是安装依赖Python 3.x的**Miniconda**。 如果已安装conda，则可以跳过以下步骤。访问Miniconda网站，根据Python3.x版本确定适合的版本。

如果我们使用Linux，假设Python版本是3.9（我们的测试版本），将下载名称包含字符串“Linux”的bash脚本，并执行以下操作：

```
# 文件名可能会更改
sh Miniconda3-py39_4.12.0-Linux-x86_64.sh -b

# -b 代表: batch mode（批量 / 自动安装模式）
```

接下来，初始化终端Shell，以便我们可以直接运行`conda`。

```
~/miniconda3/bin/conda init
```

现在关闭并重新打开当前的shell。并使用下面的命令创建一个新的环境：

```
conda create --name d2l python=3.9 -y
```

现在激活 `d2l` 环境：

```
conda activate d2l
```

安装深度学习框架和 `d2l` 软件包

在安装深度学习框架之前，请先检查计算机上是否有可用的GPU。 例如可以查看计算机是否装有NVIDIA GPU并已安装**CUDA**。 如果机器没有任何GPU，没有必要担心，因为CPU在前几章完全够用。 但是，如果想流畅地学习全部章节，请提早获取GPU并且安装深度学习框架的GPU版本。

我们可以按如下方式安装PyTorch的CPU或GPU版本：

```
pip install torch==1.12.0
pip install torchvision==0.13.0
```

我们的下一步是安装 `d2l` 包，以方便调取本书中经常使用的函数和类：

```
pip install d2l==0.17.6
```

下载 D2L Notebook

接下来，需要下载这本书的代码。 可以点击本书HTML页面顶部的“Jupyter 记事本”选项下载后解压代码，或者可以按照如下方式进行下载：

```
mkdir d2l-zh && cd d2l-zh
curl https://zh-v2.d2l.ai/d2l-zh-2.0.0.zip -o d2l-zh.zip
unzip d2l-zh.zip && rm d2l-zh.zip
cd pytorch
```

注意：如果没有安装 `unzip`，则可以通过运行 `sudo apt install unzip` 进行安装。

安装完成后我们可以通过运行以下命令打开Jupyter笔记本（在Window系统的命令行窗口中运行以下命令前，需先将当前路径定位到刚下载的本书代码解压后的目录）：

```
jupyter notebook
```

现在可以在Web浏览器中打开**http://localhost:8888**（通常会自动打开）。 由此，我们可以运行这本书中每个部分的代码。 在运行书籍代码、更新深度学习框架或 `d2l` 软

件包之前，请始终执行`conda activate d2l`以激活运行时环境。 要退出环境，请运行`conda deactivate`。

使用Jupyter Notebook

本节介绍如何使用Jupyter Notebook编辑和运行本书各章中的代码。确保你已按照 **安装**中的说明安装了Jupyter并下载了代码。如果你想了解更多关于Jupyter的信息，请参阅其**文档**中的优秀教程。

在本地编辑和运行代码

假设本书代码的本地路径为`xx/yy/d2l-en/`。使用shell将目录更改为此路径（`cd xx/yy/d2l-en`）并运行命令`jupyter notebook`。如果浏览器未自动打开，请打开http://localhost:8888。此时你将看到Jupyter的界面以及包含本书代码的所有文件夹，如 **图16.1.1**所示

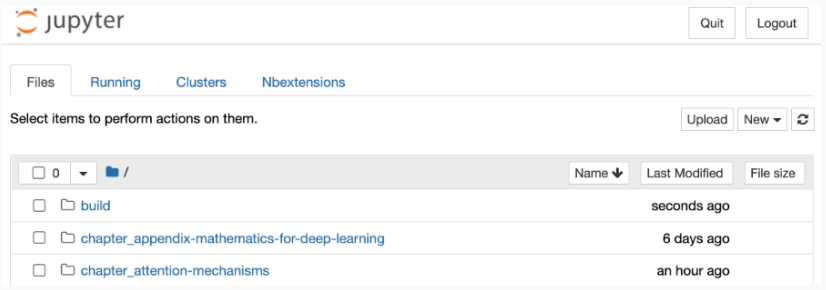

图16.1.1 包含本书代码的文件夹

你可以通过单击网页上显示的文件夹来访问notebook文件。它们通常有后缀“.ipynb”。为了简洁起见，我们创建了一个临时的“test.ipynb”文件。单击后显示的内容如 **图16.1.2**所示。此notebook包括一个标记单元格和一个代码单元格。标记单元格中的内容包括“This Is a Title”和“This is text.”。代码单元包含两行Python代码。

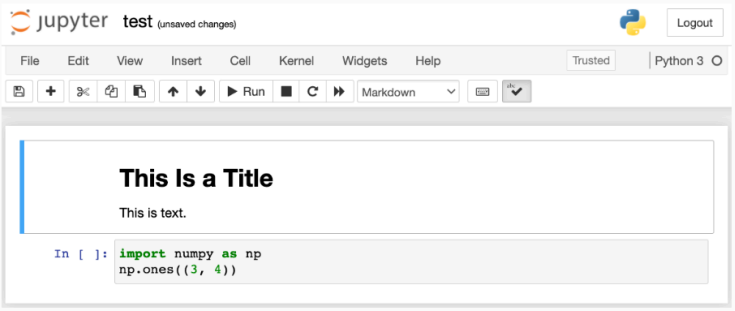

图16.1.2 “test.ipynb”文件中的markdown和代码块

双击标记单元格以进入编辑模式。在单元格末尾添加一个新的文本字符串“Hello world.”，如 **图16.1.3**所示。

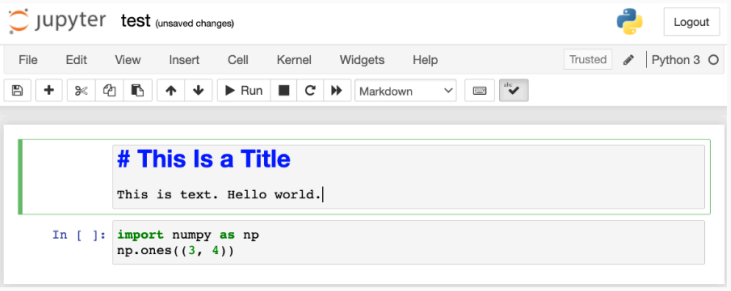

图16.1.3 编辑markdown单元格

如**图16.1.4**所示，单击菜单栏中的“Cell”→“Run Cells”以运行编辑后的单元格。

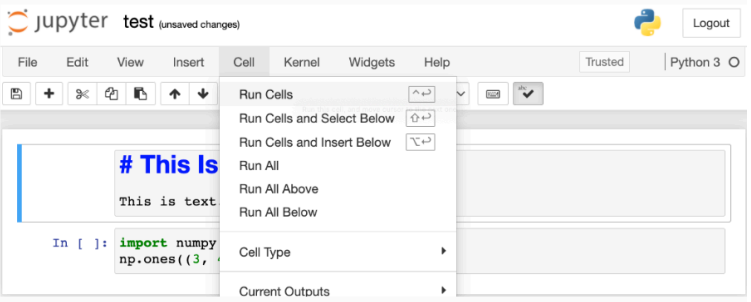

图16.1.4 运行单元格

运行后，markdown单元格如 **图16.1.5**所示。

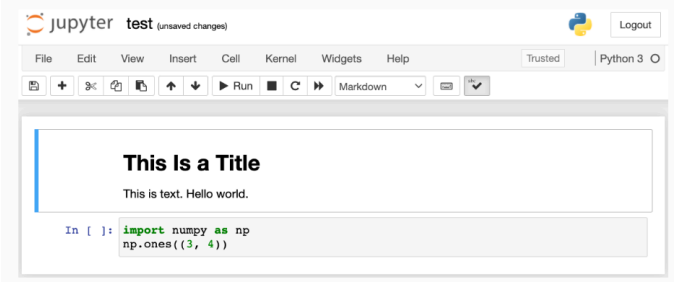

图16.1.5 编辑后的markdown单元格

接下来，单击代码单元。将最后一行代码后的元素乘以2，如 **图16.1.6**所示。

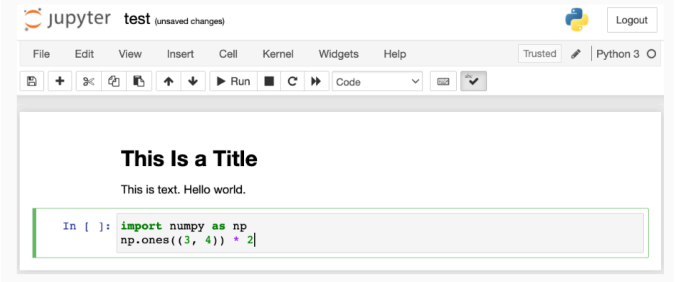

图16.1.6 编辑代码单元格

你还可以使用快捷键（默认情况下为Ctrl+Enter）运行单元格，并从 **图16.1.7**获取输出结果。

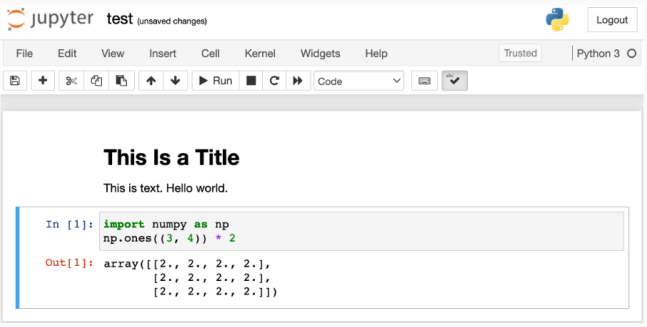

图16.1.7 运行代码单元格以获得输出

当一个notebook包含更多单元格时，我们可以单击菜单栏中的“Kernel”→“Restart & Run All”来运行整个notebook中的所有单元格。通过单击菜单栏中的“Help”→“Edit Keyboard Shortcuts”，可以根据你的首选项编辑快捷键。

高级选项

除了本地编辑，还有两件事非常重要：以markdown格式编辑notebook和远程运行Jupyter。当我们想要在更快的服务器上运行代码时，后者很重要。前者很重要，因为Jupyter原生的ipynb格式存储了大量辅助数据，这些数据实际上并不特定于notebook中的内容，主要与代码的运行方式和运行位置有关。这让git感到困惑，并且使得合并贡献非常困难。幸运的是，还有另一种选择——在markdown中进行本地编辑。

Jupyter中的Markdown文件

如果你希望对本书的内容有所贡献，则需要在GitHub上修改源文件（md文件，而不是ipynb文件）。使用notedown插件，我们可以直接在Jupyter中修改md格式的notebook。

首先，安装notedown插件，运行Jupyter Notebook并加载插件：

```
pip install d2l-notedown  # 你可能需要卸载原始notedown
jupyter notebook --NotebookApp.contents_manager_class='note
down.NotedownContentsManager'
```

要在运行Jupyter Notebook时默认打开notedown插件，请执行以下操作：首先，生成一个Jupyter Notebook配置文件（如果已经生成了，可以跳过此步骤）。

```
jupyter notebook --generate-config
```

然后，在Jupyter Notebook配置文件的末尾添加以下行（对于Linux/macOS，通常位于`~/.jupyter/jupyter_notebook_config.py`）：

```
c.NotebookApp.contents_manager_class = 'notedown.NotedownContentsManager'
```

在这之后，你只需要运行`jupyter notebook`命令就可以默认打开notedown插件。

在远程服务器上运行Jupyter Notebook

有时，你可能希望在远程服务器上运行Jupyter Notebook，并通过本地计算机上的浏览器访问它。如果本地计算机上安装了Linux或MacOS（Windows也可以通过PuTTY等第三方软件支持此功能），则可以使用端口转发：

```
ssh myserver -L 8888:localhost:8888
```

以上是远程服务器`myserver`的地址。然后我们可以使用http://localhost:8888 访问运行Jupyter Notebook的远程服务器`myserver`。下一节将详细介绍如何在AWS实例上运行Jupyter Notebook。

执行时间

我们可以使用`ExecuteTime`插件来计算Jupyter Notebook中每个代码单元的执行时间。使用以下命令安装插件：

```
pip install jupyter_contrib_nbextensions
jupyter contrib nbextension install --user
jupyter nbextension enable execute_time/ExecuteTime
```

使用Amazon SageMaker

深度学习程序可能需要很多计算资源，这很容易超出你的本地计算机所能提供的范围。云计算服务允许你使用功能更强大的计算机更轻松地运行本书的GPU密集型代码。本节将介绍如何使用Amazon SageMaker运行本书的代码。

注册

首先，我们需要在注册一个帐户https://aws.amazon.com/。 为了增加安全性，鼓励使用双因素身份验证。设置详细的计费和支出警报也是一个好主意，以避免任何意

外，例如，当忘记停止运行实例时。登录AWS帐户后，转到**console**并搜索“Amazon SageMaker”(参见 **图16.2.1**)，然后单击它打开SageMaker面板。

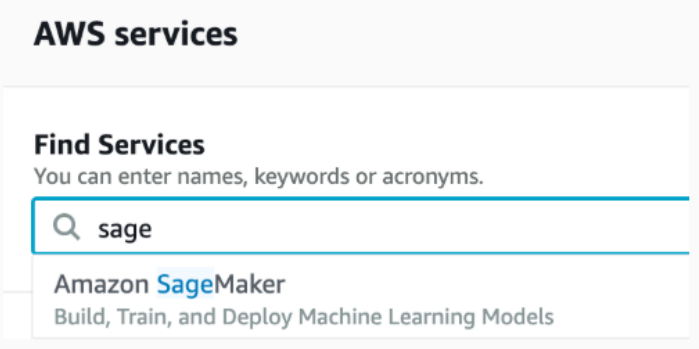

图16.2.1 搜索并打开SageMaker面板

创建SageMaker实例

接下来，让我们创建一个notebook实例，如 **图16.2.2**所示。

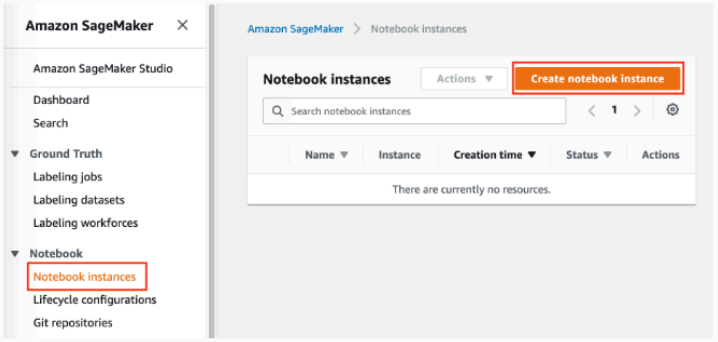

图16.2.2 创建一个SageMaker实例

SageMaker提供多个具有不同计算能力和价格的**实例类型**。创建notebook实例时，可以指定其名称和类型。在 **图16.2.3**中，我们选择 `ml.p3.2xlarge`：使用一个Tesla V100 GPU和一个8核CPU，这个实例的性能足够本书的大部分内容使用。

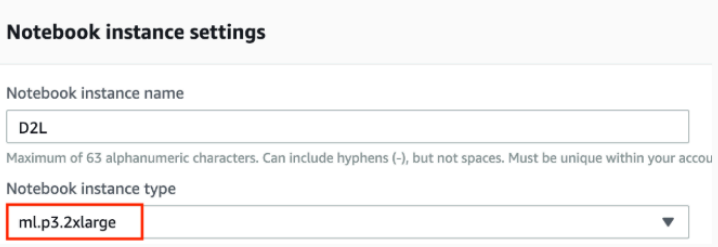

图16.2.3 选择实例类型

用于与SageMaker一起运行的ipynb格式的整本书可从https://github.com/d2l-ai/d2l-pytorch-sagemaker获得。 我们可以指定此GitHub存储库URL（**图16.2.4**），以允许SageMaker在创建实例时克隆它。

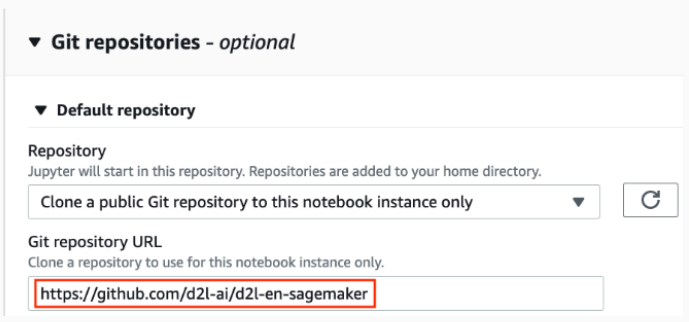

图16.2.4 指定GitHub存储库

运行和停止实例

创建实例可能需要几分钟的时间。当实例准备就绪时，单击它旁边的“Open Jupyter”链接（**图16.2.5**），以便你可以在此实例上编辑并运行本书的所有Jupyter Notebook（类似于 **16.1节**中的步骤）。

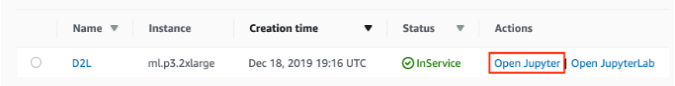

图16.2.5 在创建的SageMaker实例上打开Jupyter

完成工作后，不要忘记停止实例以避免进一步收费（**图16.2.6**）。

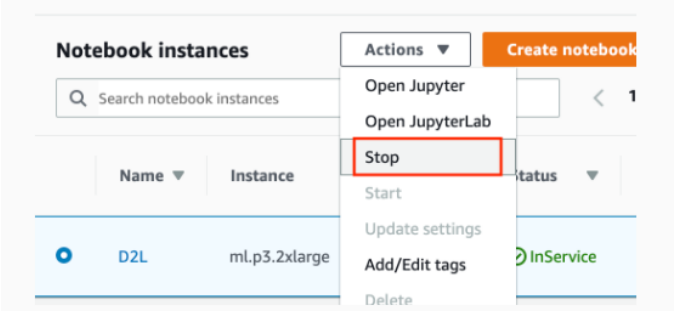

图16.2.6 停止SageMaker实例

更新Notebook

这本开源书的notebook将定期在GitHub上的**d2l-ai/d2l-pytorch-sagemaker**存储库中更新。要更新至最新版本，你可以在SageMaker实例（**图16.2.7**）上打开终端。

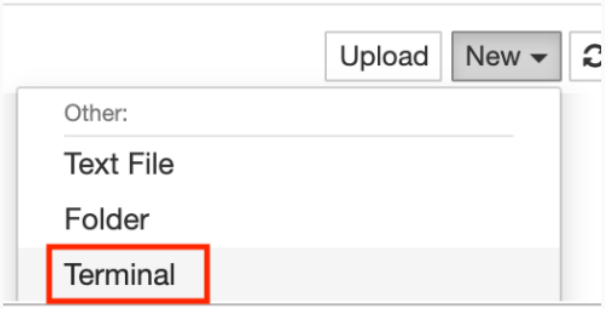

图16.2.7 在SageMaker实例上打开终端

你可能希望在从远程存储库提取更新之前提交本地更改。否则，只需在终端中使用以下命令放弃所有本地更改：

```
cd SageMaker/d2l-pytorch-sagemaker/
git reset --hard
git pull
```

使用Amazon EC2实例

本节将展示如何在原始Linux机器上安装所有库。回想一下，**16.2节**讨论了如何使用Amazon SageMaker，而在云上自己构建实例的成本更低。本演示包括三个步骤。

1. 从AWS EC2请求GPU Linux实例。
2. 安装CUDA（或使用预装CUDA的Amazon机器映像）。
3. 安装深度学习框架和其他库以运行本书的代码。

此过程也适用于其他实例（和其他云），尽管需要一些细微的修改。在继续操作之前，你需要创建一个AWS帐户，有关更多详细信息，请参阅 **16.2节**。

创建和运行EC2实例

登录到你的aws账户后，单击“EC2”（在 **图16.3.1**中用红色方框标记）进入EC2面板。

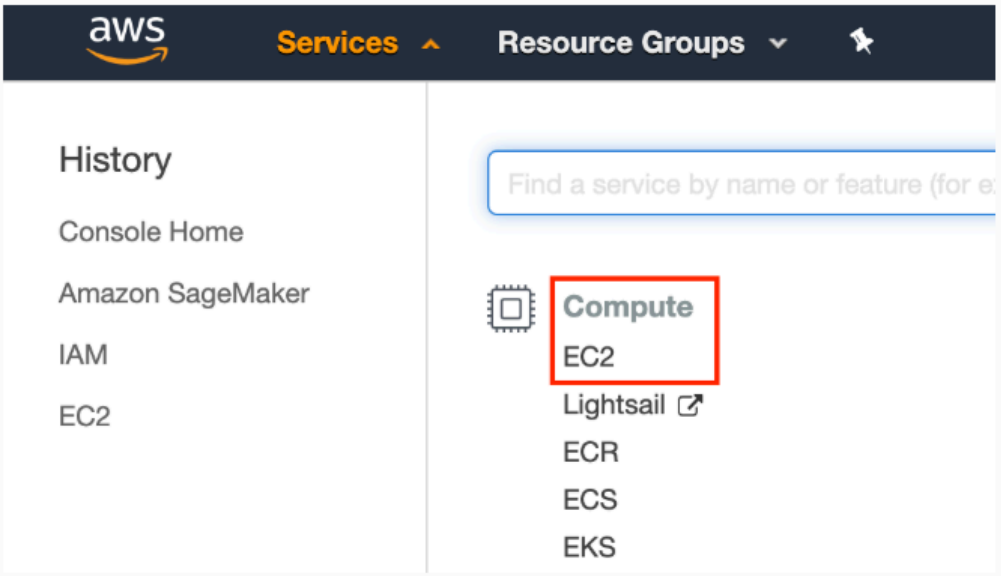

图16.3.1 打开EC2控制台

图16.3.2显示EC2面板，敏感帐户信息变为灰色。

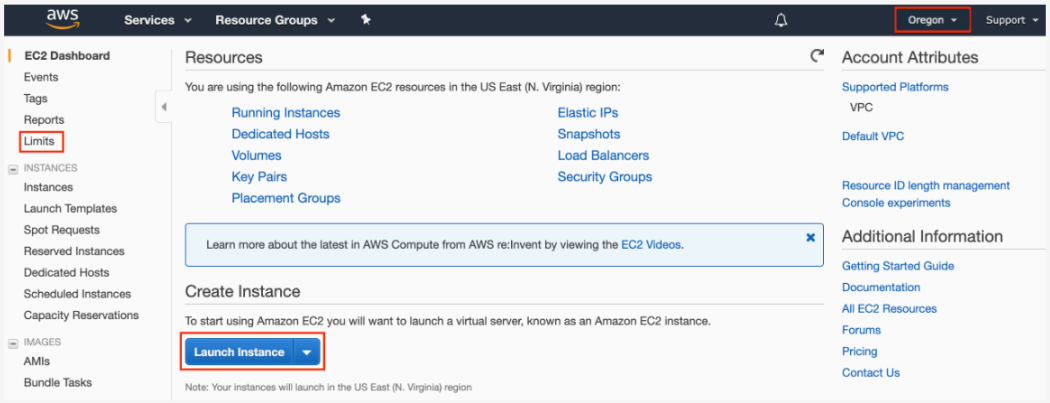

图16.3.2 EC2面板

预置位置

选择附近的数据中心以降低延迟，例如“Oregon”（俄勒冈）（**图16.3.2**右上角的红色方框）。如果你位于中国，你可以选择附近的亚太地区，例如首尔或东京。请注意，某些数据中心可能没有GPU实例。

增加限制

在选择实例之前，请点击 **图16.3.2**所示左侧栏中的“Limits”（限制）标签查看是否有数量限制。 **图16.3.3**显示了此类限制的一个例子。账号目前无法按地域打开p2.xlarge实例。如果你需要打开一个或多个实例，请点击“Request limit increase”（请求增加限制）链接，申请更高的实例配额。一般来说，需要一个工作日的时间来处理申请。

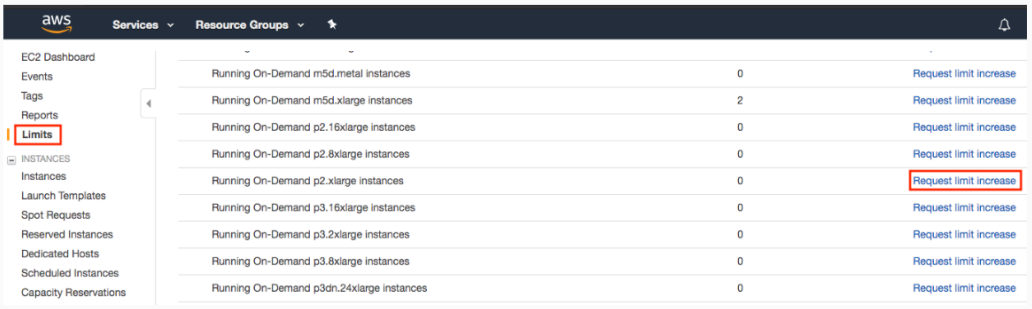

图16.3.3 实例数量限制

启动实例

接下来，单击 **图16.3.2**中红框标记的“Launch Instance”(启动实例）按钮，启动你的实例。

我们首先选择一个合适的Amazon机器映像（Amazon Machine Image，AMI)。在搜索框中输入“ubuntu”(**图16.3.4**中的红色框标记)。

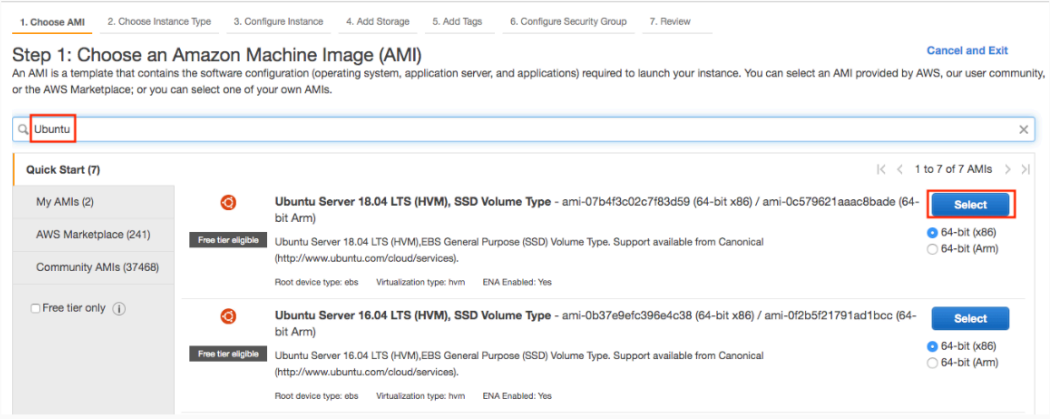

图16.3.4 选择一个AMI

EC2提供了许多不同的实例配置可供选择。对初学者来说，这有时会让人感到困惑。 **表16.3.1**列出了不同合适的计算机。

表16.3.1 不同的EC2实例类型

Name	GPU	Notes
g2	Grid K520	过时的
p2	Kepler K80	旧的GPU但Spot实例通常很便宜
g3	Maxwell M60	好的平衡
p3	Volta V100	FP16的高性能
g4	Turing T4	FP16/INT8推理优化

所有这些服务器都有多种类型，显示了使用的GPU数量。例如，p2.xlarge有1个GPU，而p2.16xlarge有16个GPU和更多内存。有关更多详细信息，请参阅**Amazon EC2 文档**。

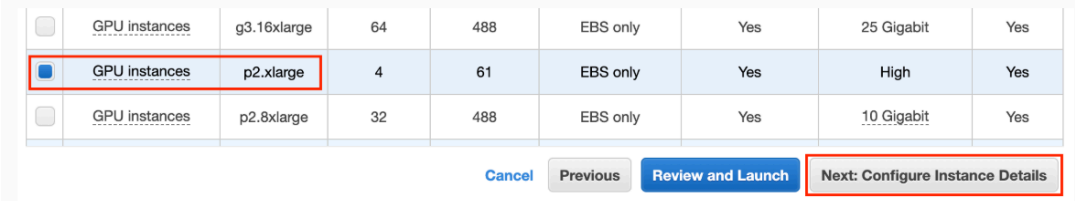

图16.3.5 选择一个实例

注意，你应该使用支持GPU的实例以及合适的驱动程序和支持GPU的深度学习框架。否则，你将感受不到使用GPU的任何好处。

到目前为止，我们已经完成了启动EC2实例的七个步骤中的前两个步骤，如 **图16.3.6** 顶部所示。在本例中，我们保留“3. Configure Instance”(3. 配置实例)、“5. Add Tags”(5. 添加标签）和“6. Configure Security Group”(6. 配置安全组）步骤的默认配置。点击“4.添加存储”并将默认硬盘大小增加到64GB(**图16.3.6**中的红色框标记)。请注意，CUDA本身已经占用了4GB空间。

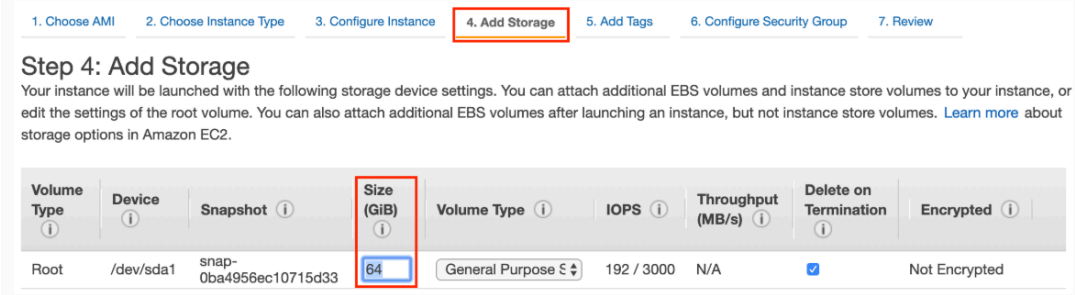

图16.3.6 修改硬盘大小

最后，进入“7. Review”(7. 查看)，点击“Launch”(启动)，即可启动配置好的实例。系统现在将提示你选择用于访问实例的密钥对。如果你没有密钥对，请在 **图16.3.7**的第一个下拉菜单中选择“Create a new key pair”(新建密钥对)，即可生成密钥对。之后，你可以在此菜单中选择“Choose an existing key pair”(选择现有密钥对)，然后选择之前生成的密钥对。单击“Launch Instances”(启动实例) 即可启动创建的实例。

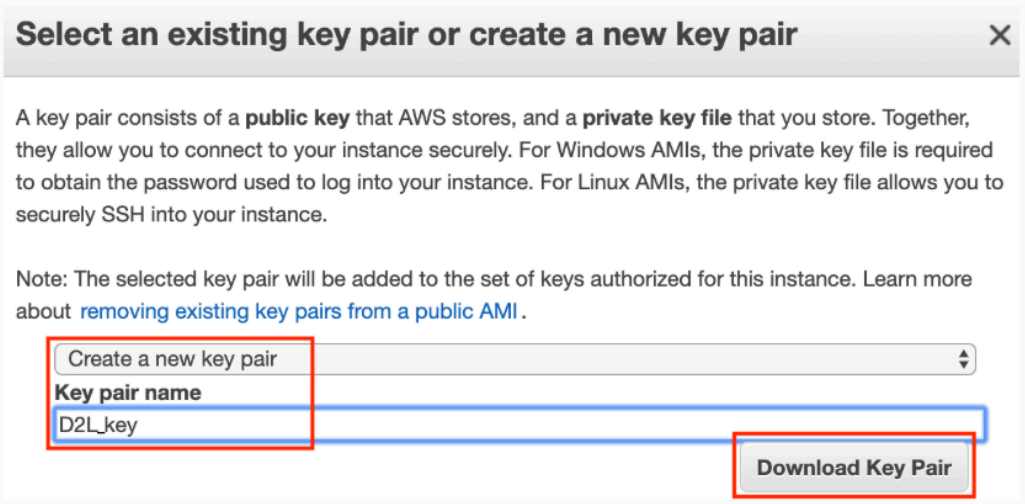

图16.3.7 选择一个密钥对

如果生成了新密钥对，请确保下载密钥对并将其存储在安全位置。这是你通过SSH连接到服务器的唯一方式。单击 **图16.3.8**中显示的实例ID可查看该实例的状态。

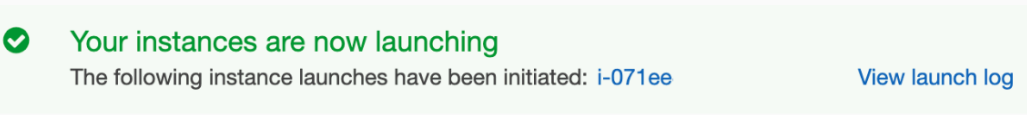

图16.3.8 单击实例ID

连接到实例

如 **图16.3.9**所示，实例状态变为绿色后，右键单击实例，选择`Connect` (连接) 查看实例访问方式。

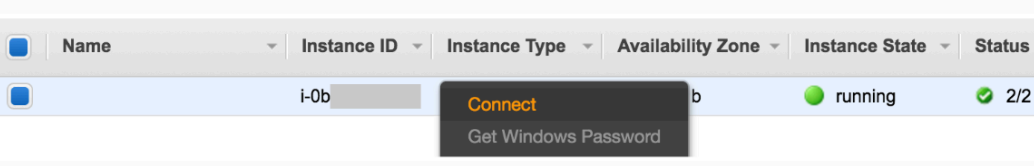

图16.3.9 查看实例访问方法

如果这是一个新密钥，它必须是不可公开查看的，SSH才能工作。转到存储`D2L_key.pem`的文件夹，并执行以下命令以使密钥不可公开查看：

```
chmod 400 D2L_key.pem
```

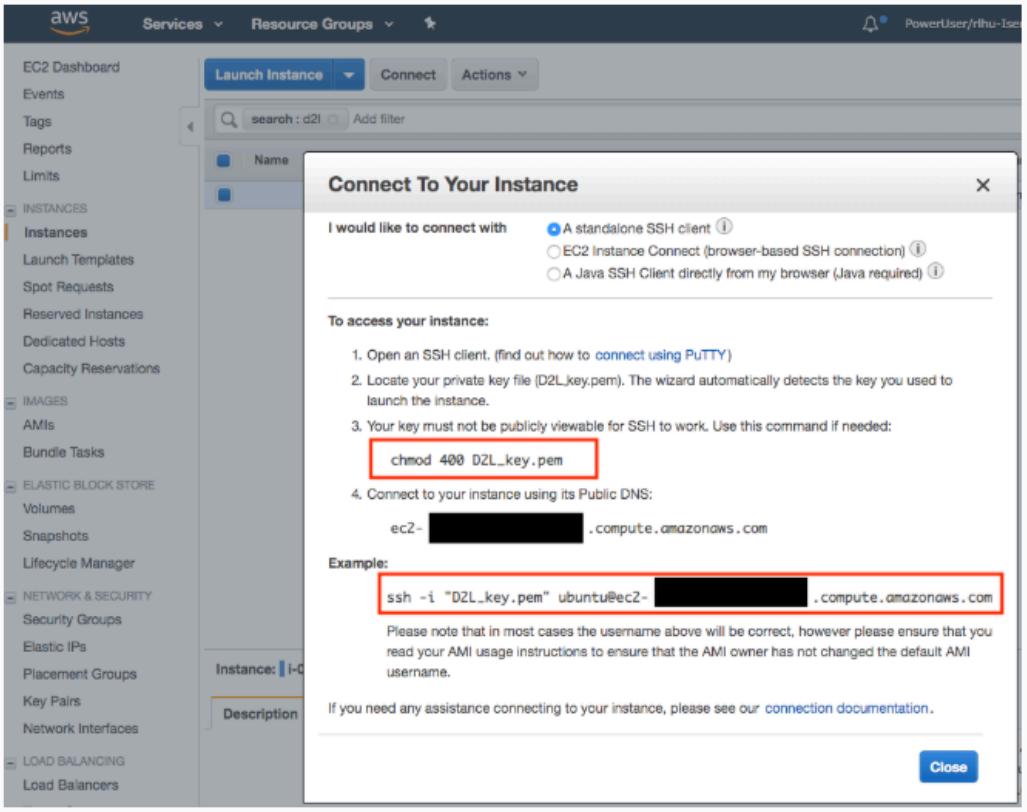

图16.3.10 查看实例访问和启动方法

现在，复制 **图16.3.10**下方红色框中的ssh命令并粘贴到命令行：

```
ssh -i "D2L_key.pem" ubuntu@ec2-xx-xxx-xxx-xxx.y.compute.am
azonaws.com
```

当命令行提示“Are you sure you want to continue connecting (yes/no)”（“你确定要继续连接吗？（是/否）”）时，输入“yes”并按回车键登录实例。

你的服务器现在已就绪。

安装CUDA

在安装CUDA之前，请确保使用最新的驱动程序更新实例。

```
sudo apt-get update && sudo apt-get install -y build-essent
ial git libgfortran3
```

我们在这里下载CUDA 10.1。访问NVIDIA的**官方存储库** 以找到下载链接，如 **图16.3.11**中所示。

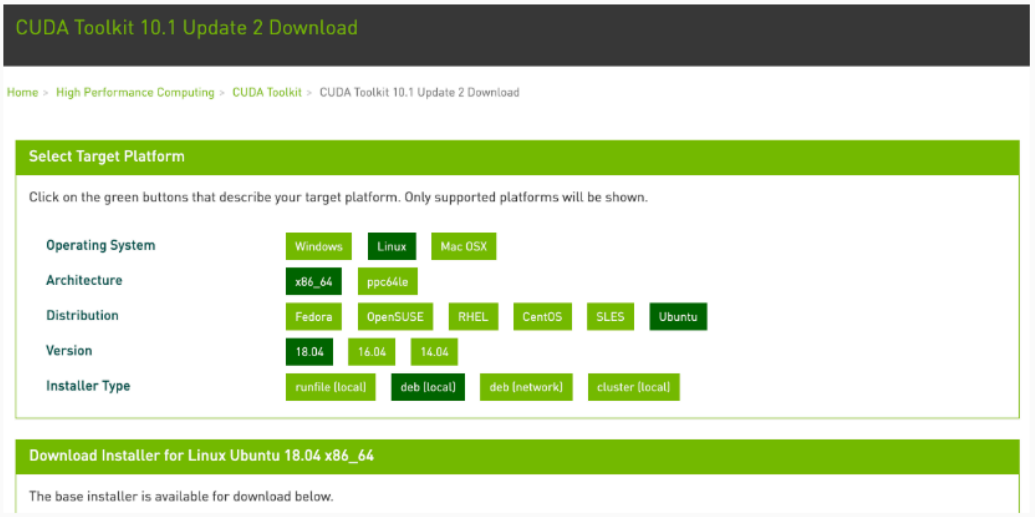

图16.3.11 查找CUDA 10.1下载地址

将说明复制粘贴到终端上，以安装CUDA 10.1。

```
# 链接和文件名可能会发生更改，以NVIDIA的官方为准
wget https://developer.download.nvidia.com/compute/cuda/repos/ubuntu1804/x86_64/cuda-ubuntu1804.pin
sudo mv cuda-ubuntu1804.pin /etc/apt/preferences.d/cuda-repository-pin-600
wget http://developer.download.nvidia.com/compute/cuda/10.1/Prod/local_installers/cuda-repo-ubuntu1804-10-1-local-10.1.243-418.87.00_1.0-1_amd64.deb
sudo dpkg -i cuda-repo-ubuntu1804-10-1-local-10.1.243-418.87.00_1.0-1_amd64.deb
sudo apt-key add /var/cuda-repo-10-1-local-10.1.243-418.87.00/7fa2af80.pub
sudo apt-get update
sudo apt-get -y install cuda
```

安装程序后，运行以下命令查看GPU：

```
nvidia-smi
```

最后，将CUDA添加到库路径以帮助其他库找到它。

```
echo "export LD_LIBRARY_PATH=\${LD_LIBRARY_PATH}:/usr/loca
l/cuda/lib64" >> ~/.bashrc
```

安装库以运行代码

要运行本书的代码，只需在EC2实例上为linux用户执行 **安装**中的步骤，并使用以下提示在远程linux服务器上工作。

- 要在Miniconda安装页面下载bash脚本，请右击下载链接并选择“copy Link address”，然后执行 `wget [copied link address]`。
- 运行 `~/miniconda3/bin/conda init`，你可能需要执行 `source~/.bashrc`，而不是关闭并重新打开当前shell。

远程运行Jupyter笔记本

要远程运行Jupyter笔记本，你需要使用SSH端口转发。毕竟，云中的服务器没有显示器或键盘。为此，请从你的台式机（或笔记本电脑）登录到你的服务器，如下所示：

```
# 此命令必须在本地命令行中运行
ssh -i "/path/to/key.pem" ubuntu@ec2-xx-xxx-xxx-xxx.y.compu
te.amazonaws.com -L 8889:localhost:8888
```

接下来，转到EC2实例上本书下载的代码所在的位置，然后运行：

```
conda activate d2l
jupyter notebook
```

图16.3.12显示了运行Jupyter笔记本后可能的输出。最后一行是端口8888的URL。

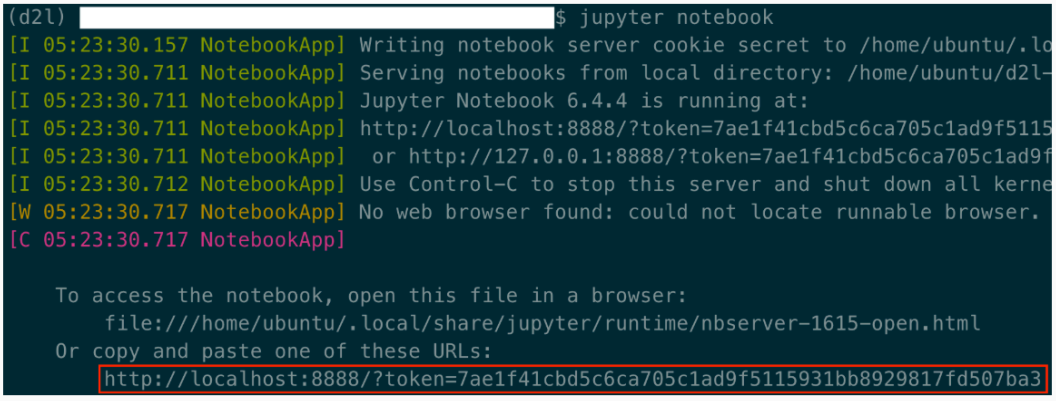

图16.3.12 运行Jupyter Notebook后的输出（最后一行是端口8888的URL）

由于你使用端口转发到端口8889，请复制 **图16.3.12**红色框中的最后一行，将URL中的“8888”替换为“8889”，然后在本地浏览器中打开它。

关闭未使用的实例

由于云服务是按使用时间计费的，你应该关闭不使用的实例。请注意，还有其他选择：

- “Stopping”（停止）实例意味着你可以重新启动它。这类似于关闭常规服务器的电源。但是，停止的实例仍将按保留的硬盘空间收取少量费用；
- “Terminating”（终止）实例将删除与其关联的所有数据。这包括磁盘，因此你不能再次启动它。只有在你知道将来不需要它的情况下才这样做。

如果你想要将该实例用作更多实例的模板，请右击 **图16.3.9**中的例子，然后选择“Image”→“Create”以创建该实例的镜像。完成后，选择“实例状态”→“终止”以终止实例。下次要使用此实例时，可以按照本节中的步骤基于保存的镜像创建实例。唯一的区别是，在 **图16.3.4**所示的“1.选择AMI”中，你必须使用左侧的“我的AMI”选项来选择你保存的镜像。创建的实例将保留镜像硬盘上存储的信息。例如，你不必重新安装CUDA和其他运行时环境。

> 配好一次 → 做成 AMI 镜像 → 删掉原实例 → 以后全用镜像新开实例

选择服务器和GPU

深度学习训练通常需要大量的计算。目前，GPU是深度学习最具成本效益的硬件加速器。与CPU相比，GPU更便宜，性能更高，通常超过一个数量级。此外，一台服务器可以支持多个GPU，高端服务器最多支持8个GPU。更典型的数字是工程工作站最多4个GPU，这是因为热量、冷却和电源需求会迅速增加，超出办公楼所能支持的范围。对于更大的部署，云计算（例如亚马逊的**P3**和**G4**实例）是一个更实用的解决方案。